Django Shared App Starter

HOW TO CREATE (JT)

```
django-admin startproject kinexis_support_app
poetry init
poetry add django
django-admin startapp kinexis_support
django-admin startapp demo
tree . ├── demo │   ├── __init__.py │   ├── admin.py │   ├── apps.py │   ├── migrations │   │   └── __init__.py │   ├── models.py │   ├── tests.py │   └── views.py ├── kinexis_support │   ├── __init__.py │   ├── admin.py │   ├── apps.py │   ├── migrations │   │   └── __init__.py │   ├── models.py │   ├── tests.py │   └── views.py ├── kinexis_support_app │   ├── __init__.py │   ├── asgi.py │   ├── settings.py │   ├── urls.py │   └── wsgi.py ├── manage.py ├── poetry.lock ├── pyproject.toml └── test_kinexis_support ├── __init__.py ├── admin.py ├── apps.py ├── migrations │   └── __init__.py ├── models.py ├── tests.py └── views.py
8 directories, 29 files
```

Django Shared App Starter Template

This is a **production-quality starter template** for a reusable Django shared app, including:

- Standalone app package
- Demo project for development
- Optional tests
- Packaging config
- Editable install support

Repo Structure

```
django-shared-example/
├── myapp/                # The reusable app
│   ├── __init__.py
│   ├── apps.py
│   ├── models.py
```

```
│   ├── admin.py
│   ├── urls.py
│   ├── views.py
│   ├── services.py
│   ├── migrations/
│ │   └── __init__.py
│   ├── templates/
│ │   └── myapp/
│ │       └── example.html
│   └── static/
│       └── myapp/
│           └── style.css
│
├── tests/                  # Optional tests
│   ├── __init__.py
│   └── test_basic.py
│
├── demo/                   # Dev project
│   ├── manage.py
│   └── demo/
│       ├── __init__.py
│       ├── settings.py
│       ├── urls.py
│       ├── asgi.py
│       └── wsgi.py
│
├── pyproject.toml
├── README.md
└── .gitignore
```

Core App Files

myapp/apps.py

```
from django.apps import AppConfig
```

```
class MyAppConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "myapp"
    verbose_name = "My Shared App"

    def ready(self):
        pass
```

myapp/models.py

```
from django.db import models


class ExampleModel(models.Model):
    name = models.CharField(max_length=255)
    created = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

myapp/admin.py

```
from django.contrib import admin
from .models import ExampleModel


@admin.register(ExampleModel)
class ExampleModelAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "created")
```

myapp/views.py

```
from django.http import HttpResponse
```

```
def health(request):
    return HttpResponse("ok")
```

myapp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("health/", views.health, name="myapp-health"),
]
```

myapp/services.py

```
from django.conf import settings


MYAPP_FEATURE_ENABLED = getattr(settings, "MYAPP_FEATURE_ENABLED", False)


def do_something():
    if not MYAPP_FEATURE_ENABLED:
        return "disabled"
    return "enabled"
```

Optional Tests

tests/test_basic.py

```
from django.test import TestCase
from myapp.services import do_something
```

```
class BasicTest(TestCase):
    def test_service(self):
        result = do_something()
        self.assertIn(result, ["enabled", "disabled"])
```

Demo Project

demo/demo/settings.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = "dev-only-secret"
DEBUG = True
ALLOWED_HOSTS = []

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "myapp",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
```

```
]

ROOT_URLCONF = "demo.urls"

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}

STATIC_URL = "/static/"
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

demo/demo/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("myapp/", include("myapp.urls")),
]
```

Packaging

pyproject.toml

```
[project]
name = "django-myapp"
version = "0.1.0"
description = "Reusable Django shared app"
requires-python = ">=3.10"
dependencies = [
```

```
    "django>=4.2",
]

[build-system]
requires = ["setuptools", "wheel"]
build-backend = "setuptools.build_meta"
```

README.md

```
# django-myapp

Reusable Django shared app.

## Dev Setup
```bash
python -m venv .venv
source .venv/bin/activate
pip install -e .
cd demo
python manage.py migrate
python manage.py runserver

## Install into another project

pip install -e ../django-myapp


or via git:

django-myapp = { git = "git@github.com:you/django-myapp.git", tag = "v0.1.0" }
```
```

.gitignore

pycache/ *.pyc .venv/ db.sqlite3 .DS_Store build/ dist/ *.egg-info/

Development Flow

```bash
cd django-shared-example
pip install -e .
cd demo
python manage.py migrate
python manage.py runserver
```

Edit `myapp/` → live reload via demo project.

Shared-App Rules

✅ Read config from `settings` ✅ No project imports ✅ No `.env` reading ✅ No project assumptions ✅ No cross-project imports

Versioning Rules

- Code change → patch
- API change → minor
- Migration change → minor/major
- Breaking change → major

If you want, I can generate this as a downloadable zip or git-ready repo layout.